

Week 2: Introduction to Serverless

Serverless Engineering Internship Guide

Week 2: Introduction to Serverless

The Complete Theoretical Intern Training & Implementation Guide

Welcome to the Week 2 Serverless Engineering Intern Guide. Building on the cloud and storage fundamentals from Week 1, this week introduces you to the serverless application paradigm. Serverless compute and database architectures enable developers to build highly scalable, event-driven applications without managing host operating systems or scaling physical servers.

This guide serves as a self-contained manual. It contains no code snippets (except for step-by-step shell execution commands, file trees, and architecture diagrams), focusing instead on explaining every serverless concept, execution lifecycle, database configuration, and integration from first principles. By reading this guide, you will understand exactly what was built during Week 2, how the components interact, and how to deploy and test the architectures step-by-step.

Table of Contents

1. Module 1: Serverless Fundamentals
2. Module 2: AWS Lambda Deep-Dive
3. Module 3: Amazon API Gateway & Request Flow
4. Module 4: AWS Identity & Security in Serverless (Execution Roles)
5. Module 5: NoSQL Persistence (Amazon DynamoDB)
6. Module 6: Serverless Authentication (Amazon Cognito User Pools)
7. Module 7: Serverless Monitoring (Amazon CloudWatch Logs & Metrics)
8. Module 8: The Student Feedback API System Architecture
9. Module 9: Terraform Resource Definitions for Serverless
10. Module 10: Hands-On Lab & Project Setup Guide
11. Module 11: Week 2 Learning Conclusion

Module 1: Serverless Fundamentals

1.1 What is Serverless?

The term "Serverless" is a misnomer; applications still run on physical servers. However, the management, patching, scaling, and provisioning of those servers are entirely handled by the cloud provider (AWS).

In traditional architectures, you run an EC2 instance that runs 24/7, costing you money even when no users are visiting your site. In a serverless architecture, your code sits idle in storage. When an event occurs (like an HTTP request or a file upload), the cloud provider instantly spins up a container, executes your code, returns the response, and shuts the container down. You are billed only for the exact milliseconds your code runs.

- Key Advantages:
- No Server Management: No OS updates, patching, or security configuration.
- Scale with Demand: Automatically scales from zero requests to thousands of concurrent executions.
- Pay-per-Execution: Never pay for idle compute time.
- High Availability: High availability and fault tolerance are built-in by default.

1.2 Event-Driven Architecture (EDA)

Serverless is built on event-driven architecture. Instead of applications running in a continuous loop checking for data, the system responds to events:

- Event Producers: Objects that generate changes in state (e.g., S3 receiving an upload, API Gateway receiving an HTTP POST request).
- Event Consumers (Handlers): Serverless functions (like AWS Lambda) that listen for events, extract the payload, execute business logic, and write to a database or trigger another service.

1.3 Stateless Computing

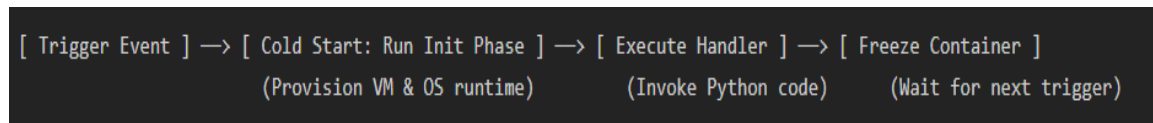
A critical requirement of serverless functions is that they must be stateless. Every execution of a Lambda function can run in a completely different, isolated container.

- You cannot save files to local disk and expect them to be there on the next execution.
- You cannot maintain persistent in-memory variables between separate requests.
- All state must be persisted externally in databases (like DynamoDB) or object storage (like S3).

Module 2: AWS Lambda Deep-Dive

2.1 Lambda Execution Lifecycle

When a Lambda function is triggered, the AWS service orchestrates three main phases:



- Init Phase (Cold Start): AWS provisions the container, launches the guest operating system, installs the language runtime (like Python or Node.js), and runs any code outside of your handler function. This phase causes a slight delay (known as a cold start).
- Invoke Phase: AWS invokes your handler function, passing it the trigger payload.
- Freeze Phase: Once the execution finishes, AWS pauses the container. If another event occurs immediately, AWS reuses the container, bypassing the Init phase (Warm Start).
- Shutdown Phase: If no events occur for a period of time, AWS destroys the container and releases its resources.

2.2 Triggers, Event Payloads, and Timeouts

- **Triggers:** Integrations that invoke Lambda. These can be synchronous (API Gateway where the client waits for a response) or asynchronous (S3 uploads, where S3 triggers the Lambda in the background).
- **Event Payloads:** When Lambda runs, it receives two dictionary objects:
- ``event``: Contains the raw payload data from the trigger (e.g., query parameters, request headers, body).
- ``context``: Contains execution metadata (e.g., remaining runtime before timeout, log group name, request ID).
- **Timeouts and Memory:**
- You allocate a specific amount of memory to a Lambda (128 MB to 10 GB). AWS scales CPU power proportionally to the allocated memory.
- You define a timeout (maximum 15 minutes). If the function runs longer than the timeout, AWS terminates the execution.

Module 3: Amazon API Gateway & Request Flow

To expose a Lambda function to the web, you need an API management layer. Amazon API Gateway acts as the front door for your backend.

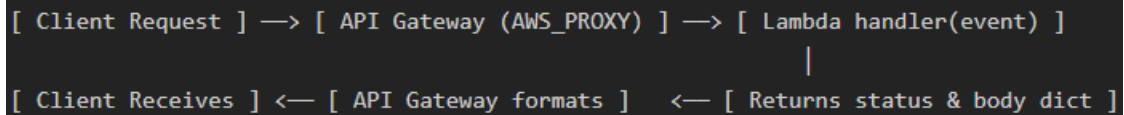
3.1 REST vs. HTTP APIs

- **REST APIs:** Feature-rich API management supporting request transformation, API keys, usage plans, and custom authorizers.
- **HTTP APIs:** Lighter, faster, and cheaper alternatives designed specifically for proxying requests directly to Lambdas.

3.2 Proxy vs. Non-Proxy Integrations

- **Non-Proxy Integration:** API Gateway parses, maps, and filters the incoming request payload using Velocity Template Language (VTL) templates before passing it to Lambda.

- Proxy Integration (AWS_PROXY): API Gateway passes the entire, raw HTTP request (headers, query parameters, body, IP address) directly to the Lambda function. The Lambda must parse the request and return a structured JSON response containing the status code, headers, and body. This is the industry-standard choice for serverless APIs because it moves request handling logic into the application code.



Module 4: AWS Identity & Security in Serverless (Execution Roles)

Every AWS Lambda function must be associated with an IAM Execution Role. This role governs the security permissions of the function itself (what other AWS resources the Lambda can interact with).

- When a Lambda function runs, it automatically assumes this role to obtain short-lived credentials.
- AWSLambdaBasicExecutionRole: A default AWS-managed policy that grants permissions to upload execution logs to Amazon CloudWatch.
- Custom S3 & DynamoDB Access: To implement the Principle of Least Privilege, you attach a policy allowing the Lambda to perform `PutItem`/`GetItem` only on the specific DynamoDB table, and `PutObject`/`DeleteObject` only on the specific S3 attachment bucket.

Module 5: NoSQL Persistence (Amazon DynamoDB)

Serverless architectures require databases that can handle highly concurrent connection bursts without crashing. Relational databases (like RDS) have strict connection limits. Amazon DynamoDB is a fully managed NoSQL database built to handle massive volumes of traffic with consistent single-digit millisecond latency.

5.1 Partition Keys and Sort Keys

Unlike SQL databases with tables, keys, and foreign relations, DynamoDB stores data as items (rows) containing attributes (columns). It uses key-value indexing:

- **Partition Key (Hash Key):** The attribute used to distribute items across physical partitions. It must be provided in every write/read operation.
- **Sort Key (Range Key):** An optional attribute used to sort items within a partition. Combining a Partition Key and a Sort Key creates a composite primary key.

5.2 Query vs. Scan Operations

- **Query:** Searches items using only the Partition Key. Highly efficient and fast, as it searches a single physical partition.
- **Scan:** Scans every partition and item in the entire table before filtering out results. This is extremely slow, CPU-intensive, and expensive on large tables. Scans should be avoided in production.

Module 6: Serverless Authentication (Amazon Cognito)

Authenticating API requests requires a secure user directory. Amazon Cognito User Pools provide a managed user directory.

6.1 Users, Clients, and Custom Attributes

- **Cognito User Pool:** A user directory in AWS that handles registration, login, multi-factor authentication, and password recovery.
- **User Pool Client:** An application integration key that allows frontends (like a web page or mobile app) to communicate with Cognito to log in users.
- **Custom Attributes:** Extra fields stored in the user profile (e.g., ``custom:role`` or ``custom:permissions``) to support role-based access control (RBAC).

6.2 Pre-Sign Up Lambda Triggers

Cognito supports custom event hooks. When a user registers, Cognito can trigger a Lambda function before the registration completes. We use a Pre-Sign Up Lambda to automatically confirm the user's account and verify their email address, bypassing the need for verification emails in development environments.

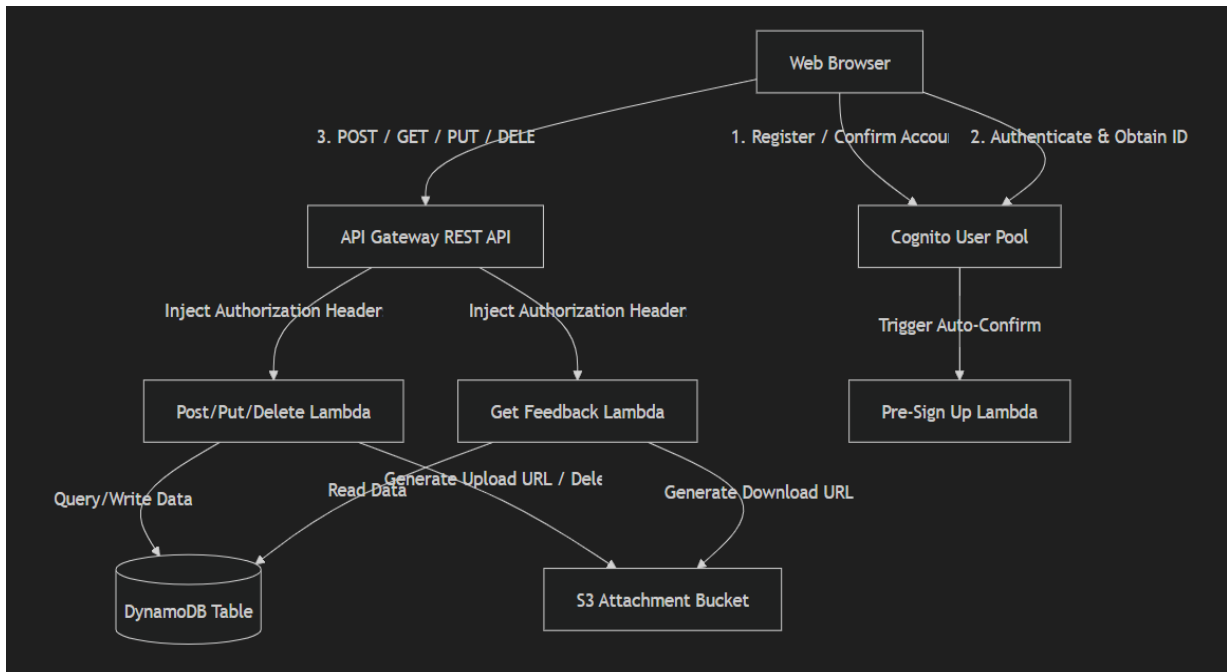
Module 7: Serverless Monitoring (Amazon CloudWatch)

Because serverless applications run in ephemeral containers, you cannot log in via SSH to read local logs. Instead, AWS automatically routes all execution outputs to Amazon CloudWatch.

- CloudWatch Logs: Every print statement or error output in Lambda is sent to a Log Stream inside a dedicated Log Group (named `/aws/lambda/<function-name>`).
- Metrics: CloudWatch tracks metrics automatically, including execution count, duration (milliseconds), execution errors, and throttling events.

Module 8: The Student Feedback API System Architecture

The project for Week 2 is a CRUD Student Feedback API. This application handles submitting feedback, uploading attachments, editing records, and checking administrator permissions.



8.1 API Endpoints

- ``POST /feedback``: Inserts a feedback record. If the user is authenticated, it links the record to their email.
- ``POST /feedback/upload-url``: Generates a pre-signed S3 upload URL, allowing clients to upload attachments directly to S3.
- ``GET /feedback``: Returns feedback records.
- If public/anonymous: Returns only the 10 most recent records.
- If logged in as a student: Returns the student's personal feedback records.
- If logged in as an administrator: Returns all feedback records.
- ``PUT /feedback``: Modifies a feedback record (only allowed if the user is the original creator).
- ``DELETE /feedback``: Deletes a feedback record (allowed for the creator or an administrator).

8.2 Authentication & Authorization Logic

API Gateway routes incoming requests to the Lambdas. The Lambdas handle authentication by reading the user's Cognito JWT token in the ``Authorization`` header:

- The Lambda extracts the ID Token, decodes the payload, and reads user attributes (`email`, `custom:role`, and `cognito:groups`).
- If `custom:role` equals `Admin`, or the user belongs to the `Admin` Cognito group, the Lambda grants administrative rights, allowing them to view and delete all feedback.
- If the user is a student, the Lambda restricts GET, PUT, and DELETE operations to only match records where `username` matches the token's `email` claim.

Module 9: Terraform Resource Definitions for Serverless

In Week 2, we define the entire serverless infrastructure using Terraform.

- `aws_dynamodb_table`: Creates the table with `type` as the partition key and `timestamp` as the sort key. It uses `PAY\_PER\_REQUEST` billing mode, which is ideal for serverless workloads.
- `aws_cognito_user_pool`: Provisions the Cognito user directory. It links `pre\_sign\_up` to our auto-confirm Lambda trigger.
- `aws_lambda_function`: Configures our Python functions, execution roles, environment variables (providing S3 bucket names and DynamoDB table names), and connects the source code zip files.
- `aws_api_gateway_rest_api`: Provisions the REST API Gateway.
- `aws_api_gateway_resource` & `aws_api_gateway_method`: Sets up paths (e.g., `/feedback`) and allowed HTTP methods.
- `aws_api_gateway_integration`: Configures the REST methods to use `AWS\_PROXY` integrations pointing to our Lambda functions.
- `aws_lambda_permission`: Explicitly allows the API Gateway service to invoke the Lambdas.

Module 10: Hands-On Lab & Project Setup Guide

This module details the directories, execution files, and configurations required to deploy and run the Week 2 serverless configurations.

10.1 Project File Directory Structure

Below is the directory tree containing the code resources for this week's assignments:

```
c:\Users\Vaibhav.S.P\Downloads\TringApps Intern
└─ Week 2/
   └─ Lambda/
      ├── main.tf                # Deploy simple HelloWorld Lambda & Gateway
      ├── lambda_function.py     # Hello world python handler
      ├── payload.json           # Mock input test payload
      └── response.json          # Execution output return log
   └─ Feedback API/
      ├── main.tf                # Full database, cognito & CRUD gateway infra
      ├── variables.tf           # Variables configuration
      ├── outputs.tf             # Outputs declaration
      ├── get_feedback.py        # Read feedback logic with JWT parsing
      ├── post_feedback.py       # Create, edit, and delete feedback logic
      ├── auto_confirm.py        # Cognito pre-signup confirmation script
      ├── index.html             # Frontend user interface webpage
      └── app.js                 # Frontend Cognito login & API request controller
```

10.2 Step-by-Step Implementation Runbook

Follow these steps to deploy and test the components of Week 2:

Task A: Basic Tools & Credentials Setup

Before beginning, ensure your local command window is authenticated with your AWS account:

- Open terminal and navigate to the project directory:

```
cd "Week 2/Lambda"
```

- Verify you have active AWS CLI credentials configured:

```
aws sts get-caller-identity
```

Task B: Simple Lambda & API Gateway Proxy Deployment

- Initialize the Terraform directory:

terraform init

- Generate the execution plan:

terraform plan -out=tfplan

- Deploy the simple Lambda and API Gateway proxy configuration:

terraform apply tfplan

- Verification: Capture the output URL printed in your terminal (e.g., `https://xxxx.execute-api.us-east-1.amazonaws.com/dev/hello`). Run a test request:

curl https://xxxx.execute-api.us-east-1.amazonaws.com/dev/hello

Expected Output: `Hello from Lambda, Vaibhav!`.

- Clean up the directory resources:

terraform destroy -auto-approve

Task C: Full Student Feedback API Deployment

Now we deploy the full CRUD application:

- Navigate to the Feedback API directory:

cd "../Feedback API"

- Initialize and deploy the infrastructure:

terraform init

terraform plan -out=tfplan

terraform apply tfplan

- Verification: Take note of the printed output variables:

1. ``api_endpoint_url`` (The REST API URL gateway).
2. ``cognito_user_pool_id`` (The ID of the user pool).
3. ``cognito_client_id`` (The app integration ID).
4. ``s3_bucket_name`` (The bucket storing attachments).

Task D: Local Testing & Frontend Verification

To run and test the frontend application:

- Open ``Week 2/Feedback API/app.js`` and verify that the configuration block matches the outputs printed in Step 3:

1. ``userPoolId`` = Output Cognito Pool ID.
2. ``clientId`` = Output Cognito Client ID.
3. ``apiBaseUrl`` = Output API Endpoint URL.

- Launch a local web server to serve the frontend (e.g., using Python):

`python -m http.server 8000`

- Open your browser and navigate to ``http://localhost:8000``.
- Register a new student account using the web form:
- Since we implemented the ``auto_confirm_lambda`` trigger, the account is automatically confirmed and verified. You do not need to wait for email confirmation.
- Log in with the registered credentials.
- Submit a feedback entry, attach a local image, and press Submit.
- Verification: Select the All Feedback tab to verify that the feedback displays.

- Open another browser tab, log in as a different student, and verify that you cannot edit or delete the first student's feedback.

Task E: CloudWatch Logging Verification

To inspect execution logs:

- Open the AWS Web Console.
- Search for CloudWatch and select Log Groups in the sidebar.
- Locate the log groups:
 - `/aws/lambda/feedback-api-get-dev``
 - `/aws/lambda/feedback-api-post-dev``
- Click on the latest log stream. Verify that the output lists the incoming SQS/Gateway event JSON and execution times.

Task F: Teardown

When you are finished testing, destroy the resources to avoid ongoing AWS charges.

- Run the destroy command:

terraform destroy -auto-approve

Module 11: Week 2 Learning Conclusion

Throughout this serverless systems engineering week, we have transitioned from traditional compute models to event-driven architectures. Here is a summary of the core competencies and learning outcomes achieved:

- Serverless Paradigms: Understood the operational and billing advantages of serverless computing, including stateless execution requirements, rapid elasticity scales, and event-driven invocation chains.

- **AWS Lambda Mechanics:** Mastered the execution container lifecycle, cold vs. warm start latency boundaries, timeouts, memory allocation, and environment variable bindings.
- **API Gateway & Routing:** Designed a complete public REST API using Lambda Proxy integrations to forward HTTP methods (GET, POST, PUT, DELETE) directly to our handlers.
- **Identity & Access Execution Roles:** Applied the Principle of Least Privilege by linking IAM roles with specific permissions policies to limit Lambda interactions with databases and S3 buckets.
- **DynamoDB Database Administration:** Built a fast, NoSQL persistence store using composite Partition Keys and Sort Keys, and optimized reads using high-performance Query requests instead of full Scan operations.
- **Managed Identity User Directories:** Built a user registration system using Cognito User Pools and User Pool Clients, and simplified local testing by utilizing a Pre-Sign Up Lambda trigger to automatically confirm user accounts.
- **CloudWatch Serverless Auditing:** Utilized CloudWatch Log Streams and Log Groups to capture Python exception traces and monitor Lambda runtime durations.

By combining these services, you have deployed a secure, production-grade CRUD database API, establishing the foundational patterns of modern serverless microservices!

Appendix A: Glossary of Week 2 Terminology

Acronym / Term	Full Name	Definition / Cloud Role
FaaS	Function as a Service	Compute model where small, stateless code blocks execute in response to events (e.g., AWS Lambda).

Acronym / Term	Full Name	Definition / Cloud Role
Cold Start	Cold Start Latency	The setup delay that occurs when AWS initializes a brand new container instance to run your Lambda function.
Proxy Integration	AWS_PROXY Integration	An API Gateway routing style that passes the raw HTTP request directly to Lambda, letting application code handle parsing.
Cognito User Pool	Cognito User Pool	A managed directory service handling user registrations, authentication tokens, and user profile metadata.
JWT	JSON Web Token	A cryptographically signed token format (Header, Payload, Signature) representing user verification claims.
Log Stream	CloudWatch Log Stream	A chronological sequence of log events originating from a single source instance (e.g., a specific Lambda execution).
Log Group	CloudWatch Log Group	A logical collection of log streams that share the same configuration (retention policies, access controls).
AssumeRole	Security Token Service (STS) role	An API action allowing an AWS service (like Lambda) to assume temporary access credentials dynamically.

Appendix B: Week 2 Troubleshooting & Debugging

Error Message / Issue	Likely Cause	Step-by-Step Resolution
Process exited before completing request	The Lambda function exceeded its configured Timeout limit, or crashed due to memory limits.	Edit main.tf, locate your aws_lambda_function, increase timeout (e.g., to 15 seconds) or memory_size (e.g., to 256MB), and deploy again.
401 Unauthorized on API routes	1. The ID Token was not passed in the request headers. 2. The prefix Bearer was omitted. 3. The token has expired.	1. Verify your request header key is Authorization. 2. Format value exactly as Bearer <ID_TOKEN>. 3. Re-authenticate via Cognito to obtain a fresh token.
Pre-signup auto-confirm failure	Cognito lacks execution permissions to invoke your auto_confirm Lambda trigger function.	Ensure the aws_lambda_permission resource in your Terraform configuration explicitly names cognito.amazonaws.com as the principal.
403 Forbidden on GET/PUT operations	The user is attempting to read or write a record that does not match their logged-in email identifier.	Check get_feedback.py or post_feedback.py. Ensure that the username claim extracted from the JWT token matches the table record ownership field.

Error Message / Issue	Likely Cause	Step-by-Step Resolution
AccessDenied writing logs to CloudWatch	The Lambda function's execution role lacks permissions to write log groups/streams.	Verify that the AWSLambdaBasicExecutionRole policy attachment is correctly bound to your Lambda execution role in your main.tf.